

Network Intrusion Detection System

Machine Learning-Based Multiclass Network Traffic Classification

Author: Madhura Kale

1. Abstract

This project develops a machine learning-based Network Intrusion Detection System (NIDS) for classifying network connections into Normal, DoS, Probe, R2L, and U2R. It uses the KDD Cup 1999 dataset, preprocesses 41 network traffic features, handles class imbalance using balanced sample weights, and compares Random Forest and XGBoost. The selected model is exposed through a Flask REST API and web interface. In the actual experiment, 494,021 raw records were reduced to 145,585 unique records after deduplication. XGBoost achieved 0.9993 accuracy and 0.9997 attack recall; both models achieved approximately 0.95 macro F1.

2. Problem Statement

Network traffic can contain malicious behavior associated with denial of service, network probing, unauthorized remote access, and privilege escalation. The objective is to automatically classify network connections as normal traffic or one of four broad attack categories.

3. Dataset

The project uses the KDD Cup 1999 dataset through scikit-learn. Each connection contains 41 input features. Original attack types are grouped into five higher-level classes. The raw dataset contained 494,021 records; 348,436 duplicates were removed, leaving 145,585 unique records.

4. Class Distribution

The cleaned data was highly imbalanced: Normal 60.33%, DoS 37.48%, Probe 1.46%, R2L 0.69%, and U2R 0.04%. Balanced sample weights were used during training to give greater importance to underrepresented classes.

5. Feature Preprocessing

The categorical features protocol_type, service, and flag are one-hot encoded with unknown categories ignored. Numerical features are standardized using StandardScaler. Preprocessing and the classifier are combined in a scikit-learn Pipeline so training and prediction use the same transformations.

6. Machine Learning Pipeline

An 80/20 stratified train-test split was used. Random Forest and XGBoost were trained with balanced sample weights. Evaluation included accuracy, custom attack recall, and standard per-class precision, recall, and F1-score. The model with the highest attack recall was selected.

7. Model Comparison

Model	Accuracy	Attack Recall	Macro F1
Random Forest	0.9993	0.9987	0.95
XGBoost	0.9993	0.9997	0.95

XGBoost was selected because it achieved the higher attack recall. Accuracy alone was not used as the selection criterion because intrusion detection places particular importance on detecting malicious traffic.

8. Random Forest Classification Report

Class	Precision	Recall	F1	Support
DoS	1.00	1.00	1.00	10,915
Probe	1.00	0.98	0.99	426
R2L	0.99	0.95	0.97	200
U2R	0.80	0.80	0.80	10
normal	1.00	1.00	1.00	17,566
Macro avg	0.96	0.95	0.95	29,117

9. XGBoost Classification Report

Class	Precision	Recall	F1	Support
DoS	1.00	1.00	1.00	10,915
Probe	0.99	1.00	0.99	426
R2L	0.97	0.97	0.97	200
U2R	0.69	0.90	0.78	10
normal	1.00	1.00	1.00	17,566
Macro avg	0.93	0.97	0.95	29,117

10. Important Interpretation

U2R is extremely rare: only 10 U2R samples appear in the test set. XGBoost achieved 0.90 U2R recall but 0.69 U2R precision. This shows why per-class metrics and support are important and why overall accuracy alone can be misleading.

11. Attack Recall

The custom `attack_recall` metric treats every non-normal category as an attack. It measures how many actual attacks were classified as any attack rather than normal. An exact multiclass error such as Probe predicted as R2L can therefore count as a detected attack for this metric, while standard classification metrics still count it as the wrong class.

12. Deployment

The selected model is served using Flask. The application provides a web interface, a `/health` endpoint, and a `POST /predict` endpoint that accepts the required network features and returns a prediction with class probabilities. Demo records are provided separately for UI testing.

13. Testing

The repository includes tests for data processing, evaluation metrics, feature preprocessing, and Flask API behavior. Local execution with `pytest` resulted in 16 passed tests.

14. Limitations

KDD Cup 1999 is an old benchmark dataset from 1999 and does not represent modern network environments, cloud infrastructure, encrypted traffic, IoT ecosystems, or contemporary attack techniques. The random train-test split can also produce optimistic results. The reported performance should therefore not be interpreted as proof of real-world or zero-day detection capability.

15. Future Improvements

Potential extensions include evaluation on a modern intrusion-detection dataset, batch prediction, SHAP-based explainability, improved monitoring/dashboard capabilities, Docker deployment, and GitHub Actions for continuous testing.

16. Conclusion

The project demonstrates an end-to-end ML workflow: dataset acquisition, cleaning, attack-category mapping, preprocessing, imbalance handling, model comparison, evaluation, API deployment, and automated testing. XGBoost was selected based on higher attack recall. The project is best presented as an educational ML engineering implementation with the KDD99 limitations clearly acknowledged.